Detecção e Mitigação Inteligente de Anomalias em Redes Definidas por Software com Grandes Modelos de Linguagem Open Source: correlação entre o Projeto de Pesquisa e o Sistema Implementado (NOC Lite)

Estudo de correlação elaborado a partir do Projeto de Pesquisa de Rodrigues (20

Artigo técnico-científico — revisão integrati

Julho de 2026

Abstract

Este artigo correlaciona o projeto de pesquisa que propõe a detecção e a mitigação inteligente de anomalias em redes definidas por software (SDN) por meio de grandes modelos de linguagem (LLMs) *open source* ajustados por PEFT/LoRA, com o sistema efetivamente implementado – o *NOC Lite*. Partindo de uma revisão da literatura estruturada em seis eixos (SDN/NFV, anomalias por ML, LLMs para NetOps, PEFT/LoRA, XAI e datasets), descreve-se a arquitetura e o pipeline de classificação do sistema construído, composto por três classificadores (heurístico, árvore de decisão e IA generativa via Google Gemini). Em seguida, apresenta-se o mapeamento artigo×sistema e a taxonomia de gaps estruturais – destacando-se a ausência de SDN, o uso de modelo fechado em vez de LLM *open source*, a inexistência de ajuste fino LoRA/QLoRA, a falta de mitigação em ciclo fechado e a dependência exclusiva de dados sintéticos. Discutem-se as convergências preservadas (baselines, tokenização, justificativa textual, métricas de latência) e propõem-se diretrizes de arquitetura para o alinhamento futuro, incluindo um ciclo fechado com LLM local, LoRA e módulo de mitigação. Conclui-se que o NOC Lite realiza parcela da intenção do projeto, mas exige evolução arquitetural substantiva nos eixos centrais para aproximar-se da proposta norteadora.

Palavras-chave: Redes Definidas por Software; Anomalias de Rede; Grandes Modelos de Linguagem; PEFT/LoRA; Inteligência Artificial Explicável.

1 Introdução

Redes de computadores modernas apresentam alta complexidade, heterogeneidade de aplicações e demandas rigorosas de desempenho, confiabilidade e segurança [?]. Nesse contexto, a identificação e a classificação de fenômenos anômalos – congestionamento transiente, falhas de enlace, degradação de desempenho – tornam-se atividades fundamentais para a operação eficiente de sistemas distribuídos [?]. Tradicionalmente, a análise de anomalias depende de intervenção manual e conhecimento especializado, o que a torna propensa a erros e com baixa agilidade de adaptação em tempo real [?]. Modelos baseados em assinaturas ou regras pré-definidas são frequentemente reativos e possuem capacidade limitada para detectar ameaças desconhecidas [?].

Nos últimos anos, grandes modelos de linguagem demonstraram capacidades de interpretação, raciocínio contextual e generalização a partir de descrições textuais e dados estruturados [?]. Diferentemente de classificadores tradicionais, tais modelos conseguem combinar múltiplas informações, lidar com incertezas e produzir não apenas uma decisão, mas também uma justificativa textual [?]. O paradigma SDN, ao separar o plano de controle do plano de dados, fornece a agilidade necessária para que agentes de IA percebam o estado da rede e executem ações de otimização ou segurança de forma autônoma [?].

O projeto de pesquisa norteador [?] propõe desenvolver um sistema autônomo baseado em LLMs *open source* (ex.: Llama, Falcon) e em ajuste fino eficiente de parâmetros (PEFT), notadamente LoRA/QLoRA, para detectar anomalias de performance em SDN e mitigá-las via reconfiguração proativa de topologia virtual (VNT) e roteamento, em ciclo fechado. Contudo, o sistema implementado – o *NOC Lite* – monitora *hosts* genéricos (endereços IP/DNS) por sondas ICMP/TCP/HTTP/DNS/SNMP, emprega o modelo fechado Google Gemini como classificador generativo e não realiza mitigação nem reconfiguração de rede.

Problema e gap. Há uma distância estrutural entre a proposta do projeto e o sistema construído. Essa distância motiva a presente análise, que tem por objetivos: (i) expor o referencial teórico das seis áreas de conhecimento envolvidas; (ii) descrever o sistema implementado e seu pipeline de classificação; (iii) apresentar o mapeamento projeto×sistema e a taxonomia de gaps; (iv) discutir convergências, limitações e ameaças à validade; e (v)

propor diretrizes de arquitetura para o alinhamento futuro. A contribuição deste artigo é, portanto, a primeira caracterização sistemática e fundamentada em evidência de código da lacuna entre uma proposta de pesquisa de LLMs *open source* para SDN e sua implementação concreta.

2 Referencial Teórico e Revisão da Literatura

A fundamentação deste trabalho articula seis eixos de conhecimento, sintetizados a partir de revisão de literatura estruturada, com 122 referências efetivamente citadas e validadas neste artigo.

2.1 Método e Corpus de Análise

A seleção do corpus seguiu um procedimento de revisão integrativa [?][?]. Partiu-se de seis áreas de conhecimento (Eixos 1–6), mapeadas a partir do Projeto de Pesquisa norteador [?] e de relatórios de análise de código do sistema NOC Lite: (1) SDN/NFV; (2) detecção e classificação de anomalias por ML tradicional; (3) LLMs *open source* e modelos de linguagem para NetOps; (4) fine-tuning eficiente de LLMs (PEFT/LoRA/QLoRA); (5) IA explicável (XAI); e (6) geração de cenários anômalos e *datasets* públicos. Como critérios de inclusão, admitiram-se obras primárias (artigos, *preprints* e relatórios técnicos) diretamente relacionadas a um dos seis eixos e com relevância para a análise arquitetural realizada; excluíram-se trabalhos fora do escopo de detectação/mitigação de anomalias em redes e fontes sem validação verificável. O corpus final compreende 122 fontes efetivamente citadas, distribuídas pelos seis eixos, consolidadas em uma única lista de referências ordenada alfabeticamente conforme a ABNT NBR 6023.

2.2 Redes Definidas por Software e Virtualização de Funções de Rede

O paradigma SDN desacopla o plano de controle do plano de dados por meio de um protocolo *southbound* padronizado (OpenFlow) [?] e de controladores programáveis (NOX [?], POX, Ryu [?], OpenDaylight [?], ONOS [?]). Essa centralização lógica cria uma única fonte de verdade e uma superfície de programação que, simultaneamente, facilita a automação e introduz novas superfícies de ataque no plano de controle [?][?][?]. A emulação (Mininet [?], Open vSwitch [?]) viabiliza a geração controlada de cenários anômalos [?]. Surveys definitivos consolidam a taxonomia e os desafios de segurança da SDN [?][?].

Complementarmente, a Virtualização de Funções de Rede (NFV), institucionalizada pelos *white papers* e pelo *framework* arquitetural da ETSI [?][?][?], e revisada por Mijumbi et al. [?][?] e Laghrissi & Taleb [?], estende o paradigma ao ciclo de vida de funções virtualizadas e sua orquestração (MANO). Sobre esse alicerce, o *Intent-Based Networking* (IBN) emerge como a camada de gestão que traduz objetivos de alto nível em configurações concretas e as garante por meio de laços fechados [?]. Trabalhos como Morales et al. [?] e a survey de Panayiotou et al. [?] antecipam a reconfiguração proativa da topologia virtual orientada a dados e ML. A detecção de anomalias em SDN transita, assim, de métodos clássicos de ML [?] para a integração de LLMs *open source* ajustados por LoRA/QLoRA.

2.3 Detecção e Classificação de Anomalias por Métodos Tradicionais

A literatura consolida que métodos estatísticos (cartas EWMA [?], Holt-Winters [?], ARIMA/SARIMA [?][?]) e classificadores de ML clássico (SVM [?], *Random Forest* [?], árvores de decisão [?], redes rasas [?]) formam o arcabouço de *baselines* sobre o qual a detecção moderna é avaliada. Surveys consagrados [?][?][?][?] reforçam que heurísticas de limiar e modelos estatísticos são insubstituíveis como referência de comparação por serem interpretáveis e leves. Para os tipos de anomalia do sistema, há corpus dedicado: DDoS é coberto por baselines de árvore/RF [?][?][?]; varredura de portas possui precedentes heurísticos e de ML [?][?]; exfiltração é abordada sobre metadados de fluxo [?]; e congestionamento/falha de enlace encontra raiz em detecção estatística de séries temporais [?][?][?].

2.4 LLMs Open Source e Modelos de Linguagem para NetOps

Surveys dedicados mapeiam a aplicação de LLMs à operação e gestão de redes (NO&M/NetOps) [?][?][?][?][?][?][?][?]. O ecossistema de modelos *open source* – LLaMA/Llama 2 [?][?], Mistral 7B [?], Falcon [?], Qwen/Qwen2.5 [?][?], BLOOM [?] e, sobretudo, Gemma [?] (derivado da tecnologia Gemini) – oferece pesos e licenças que tornam viável a substituição de modelos fechados por alternativas locais e ajustáveis [?]. A arquitetura Transformer [?], o alinhamento por RLHF [?] e o *Chain-of-Thought* (CoT) [?] fundamentam, respectivamente, a capacidade de raciocínio, de seguir instruções e de produzir justificativa textual. Aplicações concretas incluem detecção de anomalias em SDN [?][?], IBN orientado a LLM [?][?][?] e agentes de rede

autônomos [?][?][?][?][?].

2.5 Fine-Tuning Eficiente de LLMs (PEFT): LoRA, QLoRA

O ajuste fino eficiente (PEFT) – liderado por LoRA [?] e estendido por QLoRA [?], AdaLoRA [?] e DoRA [?] – é a via consagrada para adaptar LLMs *open source* a domínios específicos com orçamento de hardware modesto, reduzindo ordens de magnitude de parâmetros treináveis sem overhead de inferência [?][?][?]. Frameworks como LLM-Adapters [?] e LLaMA-Adapter [?] demonstram que LLMs 7B ajustados por PEFT igualam ou superam modelos 175B em *zero-shot*. No domínio de tráfego de rede, NetGPT [?], TrafficGPT [?] e TrafficLLM [?] mostram que LLMs podem ser ajustados (com tokenização de domínio) para entender e classificar tráfego. A base *few-shot/zero-shot* (Brown [?], Wei [?], Achiam [?]) legitima o uso de LLMs sem ajuste de pesos, mas a literatura documenta que PEFT tipicamente supera o *in-context learning* em custo e acurácia [?].

2.6 Inteligência Artificial Explicável (XAI)

A XAI é requisito fundamental para a adoção confiável de detectores em domínios de alta responsabilidade [?][?][?]. Em NLP, o survey de Danilevsky et al. [?] sistematiza como modelos de linguagem produzem explicações; Ribeiro et al. [?] (LIME) e Lundberg & Lee [?] (SHAP) oferecem atribuição local. O protótipo HuntGPT [?] integra um detector de anomalias por ML a um LLM que entrega justificativas textuais – referência explícita do projeto. Estudos recentes exploram LLMs para detecção de intrusão explicável [?][?] e explicabilidade em SDN [?]. A calibração e a confiança das justificativas são temas ativos [?][?][?][?].

2.7 Geração de Cenários Anômalos e Datasets Públicos

A literatura estabelece UNSW-NB15 [?][?] e CSE-CIC-IDS2018 [?][?] como benchmarks modernos, superando os obsoletos KDD99/NSL-KDD [?][?]. A geração dessas bases segue perfis comportamentais e emulação (Mininet, GNS3, PNETLab) [?][?]. Datasets SDN específicos (InSDN [?], SNT, DDoS2019) e técnicas de *data augmentation* (GAN/VAE/SMOTE) endereçam o desbalanceamento de classes [?][?][?]. A conversão de atributos numéricos em representações textuais – formalizada por AnoLLM [?] e revisada em surveys de LLMs em dados tabulares [?][?] – abre caminho para detectores baseados em LLMs, eixo conectado diretamente à inferência do sistema analisado [?].

3 O Sistema Implementado: NOC Lite

O *NOC Lite* é um sistema de monitoramento de disponibilidade e performance de *hosts* genéricos, estruturado como um monolito modular em FastAPI com persistência em SQLite e um *front-end* estático. A arquitetura de contêineres (Figura **??**) cobre: *front-end* (HTML/JS); API back-end (FastAPI/Uvicorn); motor de monitoramento (`asyncio`); classificador de anomalias; simulador; notificador (Telegram); e banco de dados SQLite. Atenção especial: o classificador de anomalias **não participa do laço de monitoramento** – é acionado sob demanda via `GET /api/anomalies/classify`, jamais pelo *MonitorEngine*.

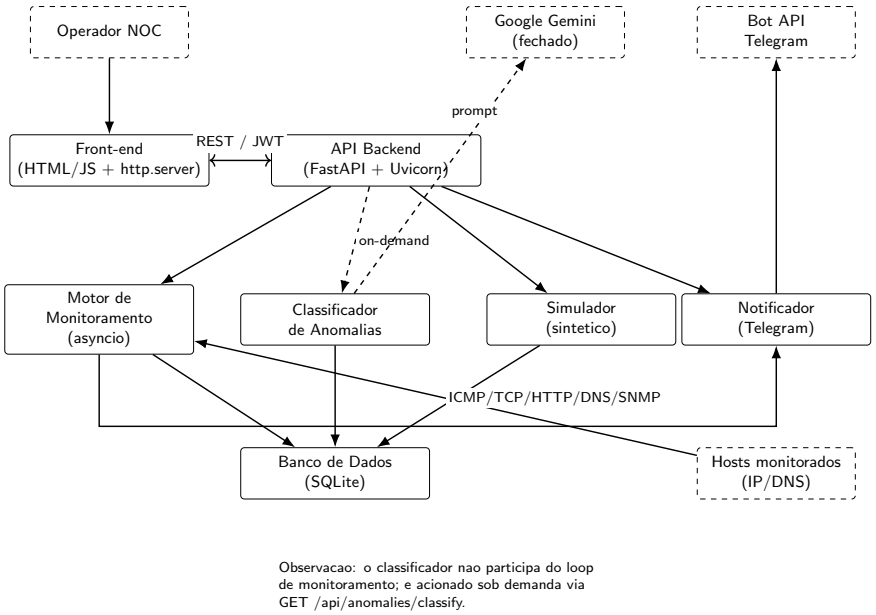


Figure 1: Arquitetura de contêineres do NOC Lite (C4, nível contêiner). Fonte: elaborada a partir de `arquitetura_sistema.md` (Dan).

3.1 Pipeline de Classificação

O subsistema de classificação expõe três estratégias sobre um conjunto fechado de seis classes – `NORMAL`, `DDOS_ATTACK`, `PORT_SCAN`, `HOST_DOWN`, `DATA_EXFILTRATION` e `NETWORK_CONGESTION` – conforme o fluxo da Figura **??**.

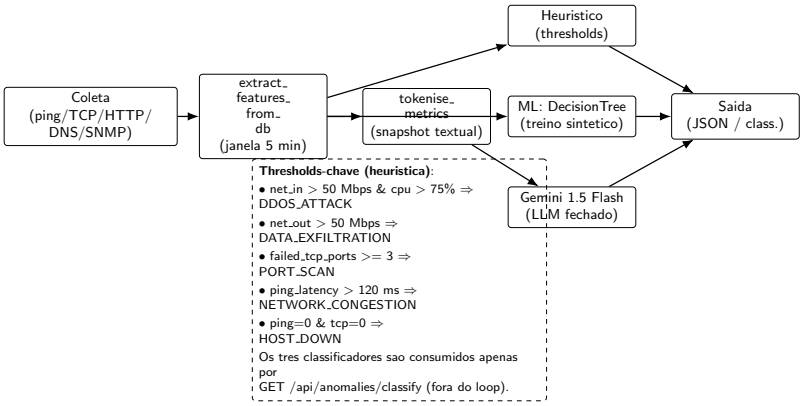


Figure 2: Pipeline de classificação de anomalias do NOC Lite: extração de features (janela de 5 min), três classificadores paralelos e tokenização para o LLM. Fonte: `notas_coder.md` (John).

1. **Heurístico (baseline 1)** – `classify_heuristic`: regras de limiar estático. Ex.: `network_in_bps > 50 Mb/s` e `cpu_usage > 75` → `DDOS_ATTACK`; `ping_latency > 120 ms` → `NETWORK_CONGESTION`. Produz confiança fixa e explicação textual.

2. **Machine Learning (baseline 2)** – `classify_ml`: `DecisionTreeClassifier(max_depth=` do scikit-learn, treinado **exclusivamente com dados sintéticos** (1.200 amostras, 200 por classe). Sem PEFT, sem LLM.

3. **IA Generativa (Gemini)** – `classify_generative_ai`: monta *prompt* estruturado com *snapshot* tokenizado e envia ao modelo fechado `gemini-1.5-flash`, retornando JSON com `classification`, `confidence` e `explanation`. Possui *fallback* para a heurística.

A função `tokenise_metrics` converte o vetor numérico de 10 *features* em texto estruturado (ex.: `PingSuccessRate: 0.95 | AvgPingLatency: 12.3 ms | CPUUsage: 34.1%`), espelhando a recomendação do projeto de converter atributos numéricos em representações textuais [?][?][?]. Esse texto é a ponte que permite ao LLM "ler" os limiares de rede em linguagem natural.

3.2 Simulador

O `anomaly_simulator.py` gera cenários anômalos sintéticos (`generate_simulated_metrics`) para as seis classes via `random.uniform`, e injeta esses vetores diretamente no banco (`run_simulated_check_cycle`), podendo popular histórico retroativo. O agendamento em tempo real usa estado em memória. Trata-se de dados 100% sintéticos, sem dependência de datasets públicos.

4 Mapeamento Projeto × Sistema e Gaps

A Tabela ?? sintetiza a correspondência entre os objetivos/componentes do projeto norteador [?] e o que o NOC Lite efetivamente entrega, com o respectivo status de alinhamento. A Figura ?? ilustra essa relação.

Table 1: Mapeamento Projeto de Pesquisa × NOC Lite.

#	**Objetivo/Componente do Projeto**	**Entrega do NOC Lite**	**Status**
1	LLM *open source* (Llama/Falcon) com fine-tuning	Gemini 1.5 Flash (fechado); Árvore de Decisão	Divergente
2	Fine-tuning PEFT (LoRA/QLoRA)	Inexistente; ML é árvore treinada do zero	Ausente
3	Ambiente SDN virtualizado (Mininet/OVS)	Hosts IP genéricos; sondas ICMP/TCP/HTTP/DNS/SNMP	Ausente
4	Mitigação via reconfiguração VNT/roteamento (ciclo fechado)	Apenas detecção + alerta Telegram; sem ação corretiva	Ausente
5	Anomalias de performance (congestionamento, falha de enlace)	Mistura anomalias de segurança/operacionais com congest. genérico	Parcial
6	Datasets públicos (UNSW-NB15, CIC-IDS2018, Synthetic SDN)	Dados 100% sintéticos (random.uniform)	Divergente
7	Baselines de comparação (heurística / ML)	Heurística + DecisionTree	Convergente
8	Tokenização de métricas em texto estruturado	`tokenise_metrics`	Convergente
9	Justificativa textual (XAI)	`explanation` (com modelo fechado)	Convergente
10	Avaliação de tempo de resposta por classificador	`latency_ms` medido por classificador	Convergente
11	Métricas resumidas (throughput, latência, jitter, perda)	SLA/jitter/tendência/health-score + SNMP	Convergente
12	Ciclo fechado coleta→classificação→mitigação	Loop aberto: coleta→severidade→alerta	Aberto

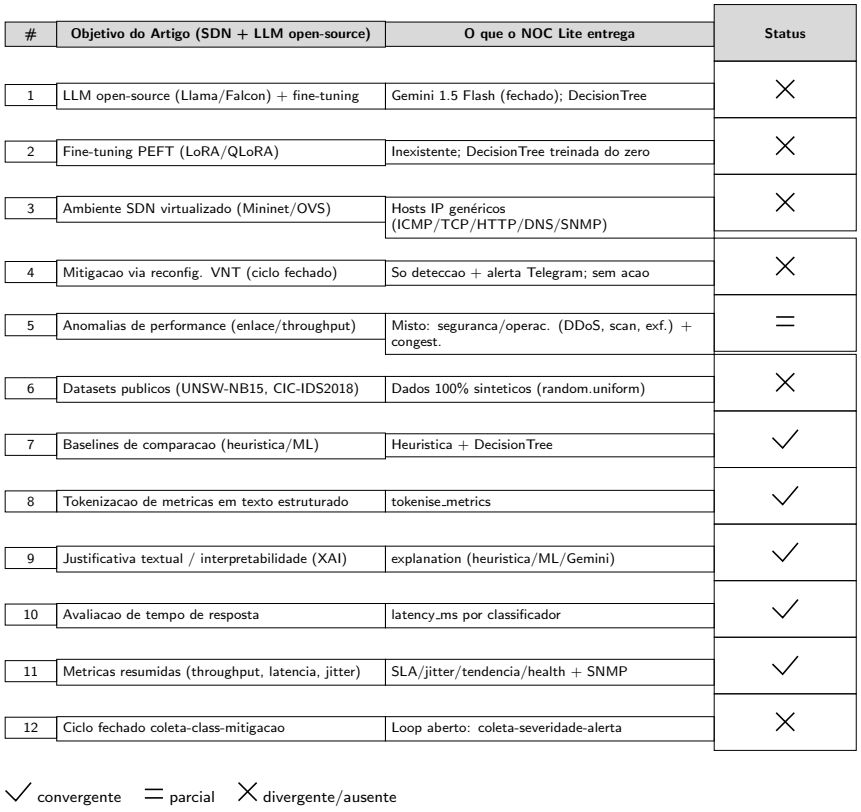


Figure 3: Mapeamento visual entre os objetivos do artigo norteador e o sistema NOC Lite. Fonte: elaborada por Chris.

4.1 Taxonomia de Gaps

A análise arquitetural identifica sete gaps, seis de severidade crítica/maior (Figura ??). Cada gap é sustentado por evidência de código.

Table 2: Taxonomia de gaps (NOC Lite × Projeto).

ID	Gap	Evidência no código / Descrição	Severidade
GAP-1	SDN vs. hosts genéricos	Busca por `sdn|mininet|vnt|controller` → 0 ocorrências; `Host.address`arbítrio	Crítico
GAP-2	Gemini fechado vs. LLM *open source*	`genai.GenerativeModel("gemini-1.5-flash")`; depende de `GEMINI_API_KEY`	Crítico
GAP-3	Ausência de LoRA/QLoRA	Busca por `lora|qlora|peft|fine.?tun` → 0; único modelo treinável é DecisionTree	Crítico

ID	Gap	Evidência no código / Descrição	Severidade
GAP-4	Ausência de mitigação / loop aberto	`scheduler.py` não referencia nenhum `classify_*`; inferência só em `/api/anomalies/classify`	Crítico
GAP-5	Dados sintéticos vs. datasets públicos	`random.uniform` em `anomaly_simulator.py`; busca `unsw|cic-ids` $\rightarrow 0$	Maior
GAP-6	Performance vs. segurança/operacionais	Classes majoritariamente de segurança; congest. por limiar genérico de latência de ping	Maior
GAP-7	Interpretabilidade com modelo fechado	`explanation` provém do Gemini; sem rastreabilidade de pesos nem reprodutibilidade local	Menor

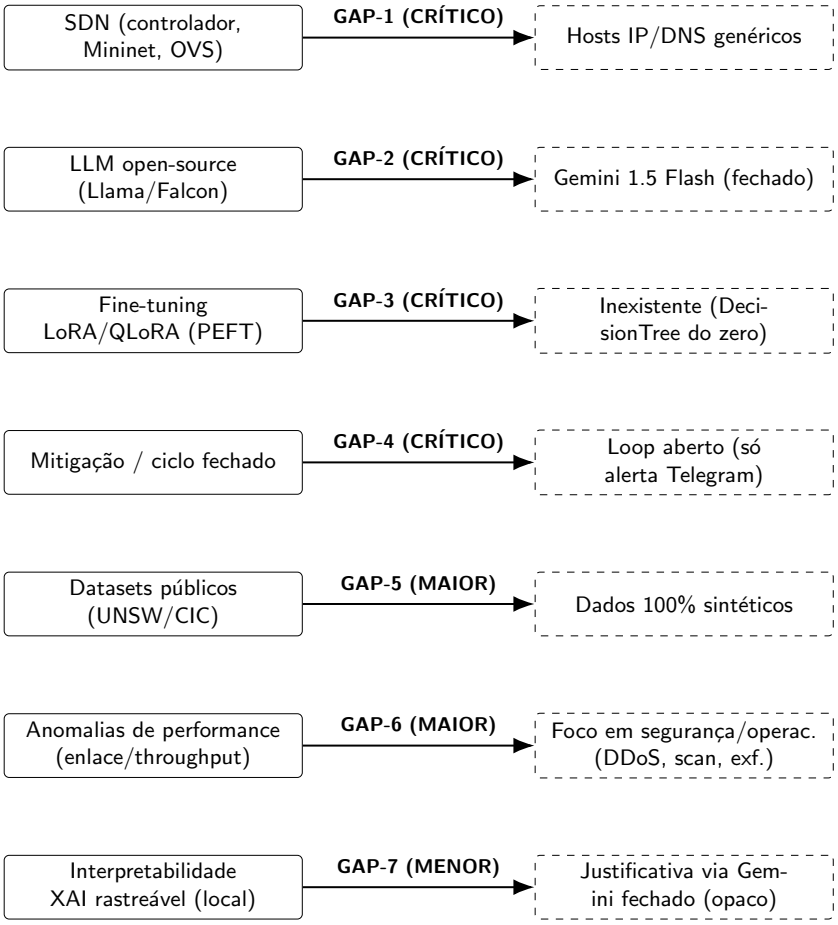


Figure 4: Taxonomia de gaps estruturais entre o projeto norteador e o NOC Lite. Fonte: elaborada por Chris.

GAP-1 (SDN vs. hosts). O projeto fundamenta toda a proposta no paradigma SDN; o NOC Lite monitora hosts IP/DNS arbitrários, sem controlador, topologia virtualizada ou planos separados [**?**]. **GAP-2 (modelo fechado).** O projeto exige LLMs *open source* e classifica explicitamente o Gemini como modelo fechado [**?**]; o sistema o usa em regime *zero-shot* [**?**]. **GAP-3 (sem LoRA/QLoRA).** A seção de PEFT do projeto não tem correspondente: o único modelo treinável é uma árvore de decisão do zero [**?**][**?**]. **GAP-4 (loop aberto).** A classificação por IA não está integrada ao laço de detecção; o ciclo observa e alerta, sem atuação autônoma [**?**]. **GAP-5 (dados sintéticos).** O treino e o *few-shot* repousam no simulador; ausente a validação cruzada com UNSW-NB15, [**?**]. **GAP-6 (escopo).** O foco do projeto é performance (congestionamento, falha de enlace, degradação de de-

sempenho); o sistema privilegia classes de segurança [?]. **GAP-7 (XAI com fechado).** A justificativa provém de modelo opaco, fragilizando a promessa de XAI do projeto [?][?].

5 Discussão

5.1 Convergências preservadas

Apesar dos desalinhamentos, o NOC Lite honra parcela da intenção do projeto:

1. **Baselines de comparação** – implementa exatamente heurística e árvore de decisão, permitindo comparação direta lado a lado [?][?].
2. **Tokenização de métricas** – `tokenise_metrics` converte o vetor numérico em representação semiestruturada, alinhada à recomendação de conversão textual [?][?].
3. **Justificativa textual / XAI** – todos os classificadores retornam `explanation`, atendendo ao objetivo qualitativo de interpretabilidade [?][?].
4. **Métricas de tempo de resposta** – o endpoint mede `latency_ms` de cada classificador, entregando a métrica de desempenho exigida [?].
5. **Métricas resumidas de rede** – SLA, jitter, tendência e health-score capturam latência/perda/jitter, coincidentes com as métricas de performance do projeto [?].

5.2 Limitações e ameaças à validade

Viés de dados sintéticos. O classificador de ML é treinado 100% com o próprio simulador, que compartilha os mesmos limiares da heurística; a árvore aprende essencialmente a "copiar" a heurística em forma de árvore, com alto desempenho no sintético e provável colapso em produção [?][?]. **Confiança não calibrada.** A confiança do Gemini é autorreportada e a do ML é probabilidade de folha não calibrada [?]; a comparação entre classificadores é, portanto, metodologicamente assimétrica. **Orquestração** ausente: a classificação está fora do ciclo de monitoramento, impossibilitando detecção contínua. **Validade externa**: a ausência de datasets públicos impede a comparação com o estado da arte [?][?].

Ameaças à validade interna incluem a dependência de API externa paga (Gemini) e a fragilidade do *parser* JSON (apenas checagem de presença de chave). **Ameaças à validade de construção**: os limiares de 50 Mb/s e 120 ms são fixos e universais, sem *baseline* por host/link, gerando falsos positivos em enlaces de capacidades distintas [**?**].

6 Diretrizes de Arquitetura para Alinhamento

Para reduzir a distância entre o NOC Lite e o projeto, propõe-se uma evolução arquitetural incremental, respeitando o que já existe e fechando os sete gaps. O ponto central é a transição de um *loop aberto* para um *ciclo fechado* com LLM *open source*, LoRA e mitigação, conforme a Figura **??**.

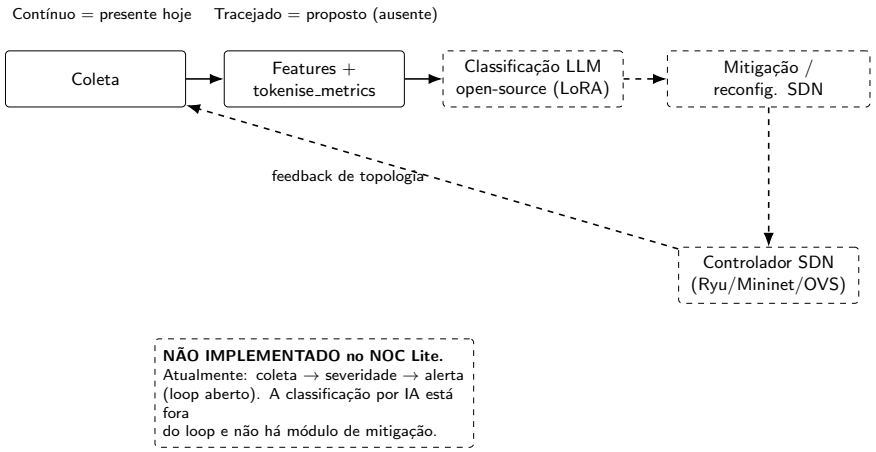


Figure 5: Arquitetura proposta de ciclo fechado: coleta (SDN + hosts) → tokenização → classificação (LLM *open source* + LoRA) → mitigação (módulo de ação) → reconfiguração via controlador. Fonte: elaborada por Chris.

1. **Substituição do modelo fechado por LLM local (fecha GAP-2, GAP-3).** Introduzir contêiner de inferência local (vLLM/llama.cpp/Ollama) servindo Llama 3 (7–13B) ou Falcon via endpoint compatível com OpenAI; substituir `gemini-1.5-flash` por cliente HTTP local, mantendo o *fallback* heurístico. Aplicar LoRA/QLoRA (rank 16–32, alpha 32) sobre o modelo base, reutilizando os *prompts* de `tokenise_metrics` [**?**][**?**][**?**].

2. **Pipeline de dados com datasets públicos (fecha GAP-5).** Criar módulo `data_ingest` que baixa/normaliza UNSW-NB15, CSE-CIC-IDS2018 e Synthetic SDN para o *schema* de *features*; treinar LoRA e

a árvore com dados reais, preservando o simulador para demonstração [?][?].

3. **Ambiente SDN e fechamento do ciclo (fecha GAP-1, GAP-4).** Adicionar contêiner Mininet + controlador Ryu que emule a topologia e exponha telemetria de enlace/fluxo; generalizar `Host`/`CheckResult` para telemetria de fluxo SDN; criar `mitigation_engine` que mapeia `classification` → ação corretiva (re-rota, QoS, *drop* de fluxo) via NorthBound API do controlador; integrar o classificador dentro do `MonitorEngine`, fechando o ciclo [?][?][?].

4. **Realinhamento do catálogo de anomalias (fecha GAP-6).** Separar claramente performance SDN (congestionamento de enlace, falha de enlace, degradação de throughput/jitter) de segurança/operacional (DDoS, port scan, exfiltração, host down), priorizando a primeira família [?].

5. **Preservação dos pontos fortes.** Manter `tokenise_metrics`, o par baseline heurístico + ML, a medição de `latency_ms` e o formato de `explanation` como base do novo pipeline [?][?].

6. **Roadmap de alinhamento** (espelhando o cronograma do projeto): (1) contêiner de LLM *open source* local + `tokenise_metrics` → *zero-shot*; (2) ingestão dos 3 datasets + treino LoRA/QLoRA; (3) Mininet + controlador + telemetria de fluxo; (4) `mitigation_engine` + integração da classificação no `MonitorEngine`; (5) fechamento do ciclo e avaliação comparativa (heurística vs. ML vs. LLM *open source* com/sem LoRA), reutilizando `latency_ms` e acrescentando acurácia/consistência.

7 Conclusão

O presente artigo correlacionou o projeto de pesquisa que propõe detecção e mitigação de anomalias em SDN com LLMs *open source* ajustados por PEFT/LoRA [?] com o sistema efetivamente implementado, o NOC Lite. Evidenciou-se que o sistema realiza com sucesso a parcela de *baselines* e de representação textual do projeto – heurística, árvore de decisão, tokenização de métricas, justificativa textual e medição de latência – mas diverge estruturalmente nos eixos centrais da proposta. Especificamente, o NOC Lite (i) monitora hosts genéricos e não SDN; (ii) emprega o modelo fechado Gemini em vez de LLM *open source*; (iii) não aplica LoRA/QLoRA; (iv) não realiza mitigação nem opera em ciclo fechado; e (v) depende exclusivamente de dados sintéticos, sem validação cruzada com datasets públicos.

Tais gaps não são ajustes pontuais: exigem evolução arquitetural substantiva, sintetizada nas diretrizes de um ciclo fechado com LLM local, LoRA e módulo de mitigação. As convergências identificadas funcionam como alicerce seguro sobre o qual construir essa evolução, uma vez que o `tokenise_metrics`, os baselines e a medição de latência já cumprem objetivos específicos do projeto. Como trabalho futuro, recomenda-se a implementação do `mitigation_engine` e a condução do experimento comparativo head-to-head entre LLM *open source* ajustado por QLoRA e LLM fechado *zero-shot* sobre o mesmo conjunto de métricas de rede – lacuna exatamente apontada pela literatura [**?**][**?**].

Referências